

Illegal Brick Kiln Detector

AI-assisted satellite review for environmental monitoring in Bangladesh

Team PTSD

AI Hackathon SciBlitz AI Challenge 2026

Samonwita Sarker (Leader), Protik Dev, Rotna Dipika Debnath, Touhidul Alam Seyam

July 2026

Summary. Illegal Brick Kiln Detector is a map-first web app for finding likely brick kilns in satellite tiles. It uses a fine-tuned YOLO11n-OBb model, served through a Python prediction endpoint, and a Next.js dashboard for review. The reviewer sees the tile, class, confidence, location, and oriented box before deciding whether a detection is worth follow-up.

1 Problem statement

Brick kilns are a local pollution and land-use problem in Bangladesh. They are spread across many districts, and in satellite images they can look similar to other industrial or rural structures. Manual review takes time, especially when a reviewer has to move between maps, image crops, and notes.

A useful tool cannot just say "kiln" or "no kiln." It has to show where the object is, how confident the model is, and what image evidence the model used. This project targets environmental analysts, civil-society monitors, and government reviewers who need a faster way to triage suspected kilns before any field check or enforcement step.

2 Proposed solution

The project is an AI-assisted dashboard for district-level kiln review. A reviewer selects a district, runs prediction on seeded satellite tiles, then reviews the detections on a map and in a list. Each detection includes latitude, longitude, kiln class, confidence, source tile, and an oriented bounding box in the detail view.

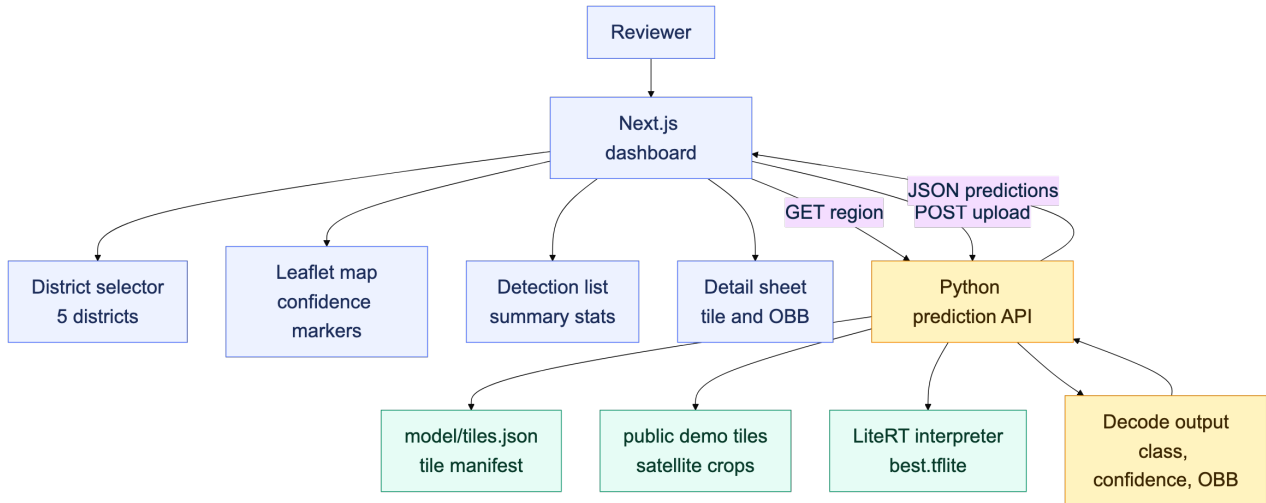


Figure 1: Application architecture for the dashboard, prediction API, model file, tile manifest, and review UI.

The diagram separates the user interface from the model-serving layer. The dashboard handles district selection, map review, detection lists, and the detail sheet. The Python API loads seeded tiles or uploaded images, runs the LiteRT model, decodes the oriented boxes, and sends structured JSON back to the UI.

Implemented product surface.

- Next.js App Router dashboard with shadcn/ui components and Leaflet mapping.
- Python `/predict` API supporting seeded region prediction and transient image upload prediction.
- LiteRT/TFLite model artifact at `model/best.tflite` for local or Vercel-style Python serving.

- Five supported Bangladesh districts: Brahmanbaria, Jessore, Manikganj, Mymensingh, and Tangail.
- Fifty bundled demo tiles, ten per district, listed in `model/tiles.json` and served from `public/demo-tiles/`.

3 Methodology

The training workflow is in `yolo-obb-n (1).ipynb`. The app workflow is in the prediction API, typed frontend parser, region helpers, and dashboard components in this repository.

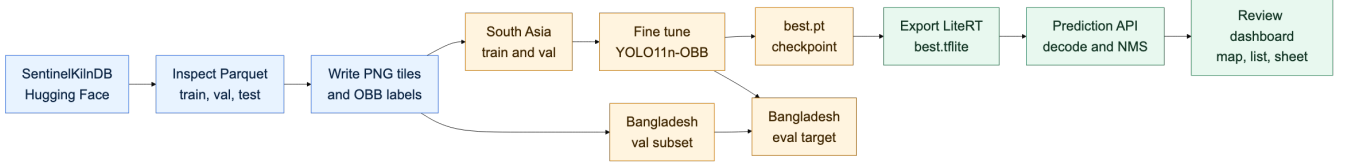


Figure 2: Training and deployment path from SentinelKilnDB to the LiteRT model used by the app.

The pipeline starts with SentinelKilnDB Parquet files and turns them into YOLO-OBB training data. The model is trained on the wider South Asia split, while the Bangladesh subset is kept for regional evaluation. The trained `best.pt` checkpoint is then exported to `best.tflite` so the app can run inference through a lighter Python API.

Data preparation. The notebook downloads SentinelKilnDB from the Hugging Face dataset repository SustainabilityLabIITGN/SentinelKilnDB. It reads the train, validation, and test Parquet files, extracts PNG tile images, and writes YOLO-OBB label files. The final class order is CFCBK, FCBK, and Zigzag.

Training split and regional evaluation target. The notebook trains on the full South Asia split instead of Bangladesh only. That gives the model more kiln examples and keeps the rare CFCBK class from becoming too small. It also creates a Bangladesh validation subset from latitude 20.5 to 26.7 and longitude 88.0 to 92.7.

Product workflow. Predictions are transient. The dashboard requests them, renders markers and list rows, and opens a detail sheet for visual review. Upload prediction accepts an image data URL in memory and does not save the image.

4 AI/ML approach

The detector starts from the pretrained Ultralytics `yolo11n-obb.pt` checkpoint and fine-tunes it for oriented object detection. Oriented boxes matter here because kilns are often long, narrow, and rotated inside the tile. The deployed model predicts CFCBK, FCBK, and Zigzag.

Item	Value from notebook or repository
Base model	<code>yolo11n-obb.pt</code> , pretrained Ultralytics YOLO OBB model
Task	Oriented object detection for brick kiln classes
Dataset	SentinelKilnDB, downloaded from Hugging Face SustainabilityLabIITGN/SentinelKilnDB
Image size	128 px training and validation tiles in the completed notebook run
Epochs	50 completed epochs
Batch	256 on Kaggle GPU environment
Device	Kaggle CUDA run with Tesla T4 GPUs shown in notebook output
Augmentation	90 degree rotation, horizontal and vertical flips, scale, translate, mosaic, cosine LR
Export	<code>model/best.pt</code> to LiteRT/TFLite <code>model/best.tflite</code> ; expected shape input (1,3,256,256), output (1,8,1344)

Table 1: Model and training configuration.

The Python API converts each image to RGB, resizes it to the model input size, normalizes pixels, runs the LiteRT interpreter, decodes YOLO OBB output, filters by confidence, applies NMS, maps tile coordinates to latitude and longitude, and returns JSON to the frontend.

5 Results

The completed notebook run extracted 71,856 training tiles and 23,952 validation tiles. Ultralytics reported 24,642 training background images and 8,214 validation background images during label scanning. The Bangladesh validation subset contained 3,929 tiles. The saved notebook output includes final overall validation metrics, but not a separate final Bangladesh-only table, so this report treats that subset as an evaluation target rather than a reported score.

Class	Images	Instances	Precision	Recall	mAP50	mAP50-95
All	23,952	21,042	0.651	0.627	0.643	0.359
CFCBK	587	649	0.661	0.572	0.592	0.374
FCBK	9,010	11,339	0.618	0.672	0.634	0.330
Zigzag	7,110	9,054	0.675	0.636	0.702	0.375

Table 2: Final validation metrics recorded in the notebook output for `best.pt`.

System results. The repository has a working prediction API contract and frontend parser. A region request returns `region`, `generatedAt`, and predictions with `id`, latitude, longitude, confidence, label, tile URL, optional class name, and optional OBB points. The UI summarizes total detections, confidence bands, average confidence, and per-class counts.

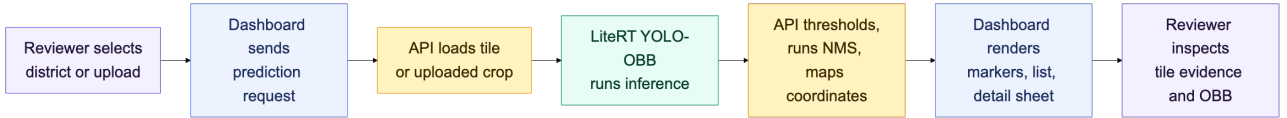


Figure 3: Reviewer interaction sequence from district selection to evidence inspection.

This flow follows one review run. The reviewer starts with a district or upload, the frontend calls `/predict`, the API loads image evidence and runs model inference, then the decoded predictions return to the dashboard. The reviewer sees the result as map markers, list rows, and a detail sheet rather than a raw model response.

6 Limitations and future work

Current limitations.

- The demo uses fifty bundled tiles and does not yet scan arbitrary large satellite mosaics.
- The available notebook output reports overall validation metrics, while Bangladesh-specific final metrics were not present in the saved cell outputs.
- Uploaded image predictions are transient and use reference tile metadata for approximate coordinates when the upload has no georeference.
- The API uses axis-aligned IoU during NMS even though the model predicts oriented boxes. This is practical, but it can handle edge cases imperfectly.
- There is no durable review database, audit log, reviewer feedback loop, or active-learning queue yet.

Future work.

- Add a geospatial tiling pipeline for wider Bangladesh coverage and scheduled scans.
- Report Bangladesh-only precision, recall, mAP, and confusion analysis as a required evaluation artifact.
- Add reviewer feedback persistence so confirmed false positives and true positives can improve future training.
- Calibrate thresholds by district or kiln class. Triage may need higher recall, while enforcement review needs higher precision.
- Add exportable evidence bundles with source tiles, OBB overlays, and metadata.

Reproducibility. Install app dependencies with `bun install`. Run local development with `bun run dev` for Next.js and `python api/predict.py` for the prediction endpoint. The notebook writes `/kaggle/working/best.pt`; the export script converts it to `model/best.tflite`. Validate with `bun run lint` and `bun run build`.